Technical Presentation

March 26, 2026

Outline

What is Hadoop?

Apache Hadoop is an open-source framework for **distributed storage** and **large-scale data processing**.

- Created by **Doug Cutting** (2006, Yahoo!)
- Inspired by Google papers (GFS + MapReduce)
- Designed to run on **commodity hardware**
- Handles **petabytes** of data

Hadoop

2006 – present
Version 3.x
Apache 2.0 License
Written in Java

Why Hadoop?

The Problem

Data is growing exponentially: logs, IoT sensors, social media, transactions...
A single server can no longer store or process everything.

The Hadoop Solution

- **Scale-out**: add more machines rather than buying a bigger one
- **Fault tolerance**: data is automatically replicated
- **Low cost**: runs on commodity hardware
- **Data locality**: computation moves to the data (not the other way around)

The 3 Pillars of Hadoop

Where is the data?

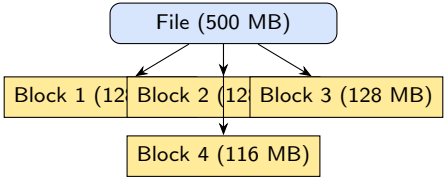
Who manages the tasks?

How to process?

HDFS – Core Principles

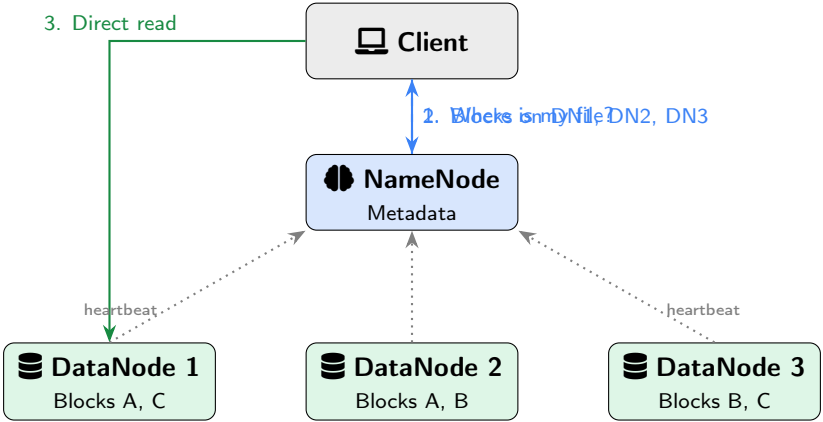
HDFS = Hadoop Distributed File System.

- Files are split into **blocks** (128 MB by default)
- Each block is **replicated 3 times** across different machines
- Optimized for **sequential reads** of large files
- **Write-once, read-many** model



Each block is replicated
3 times on different nodes

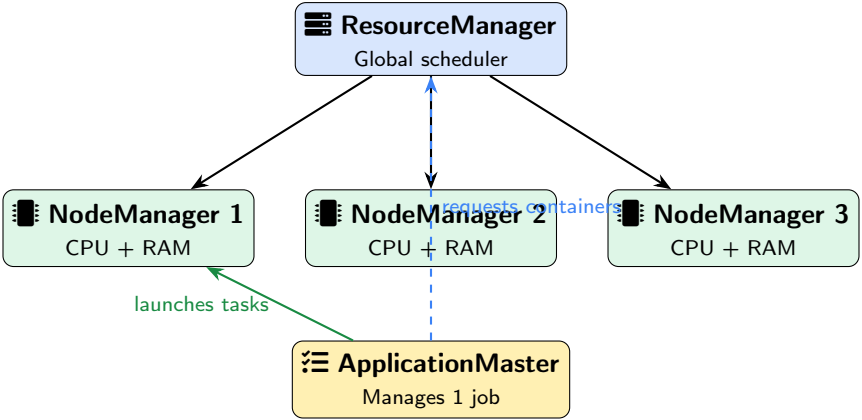
HDFS – NameNode / DataNode Architecture



HDFS Roles

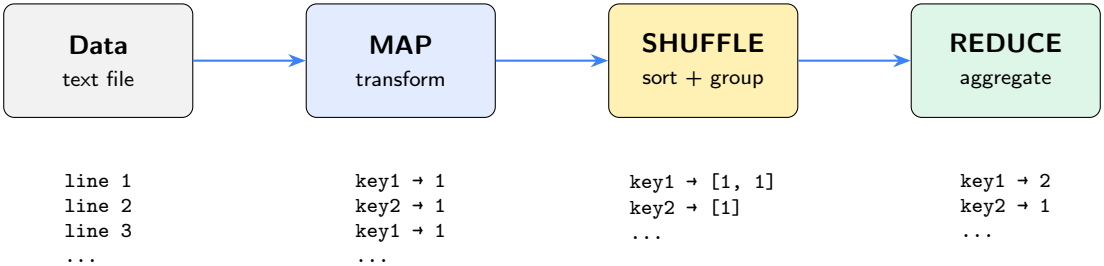
Component	Role
NameNode	Master server. Stores **metadata**: file tree, block locations, permissions. Does **not store any actual data**.
DataNode	Worker server. Stores **data blocks** on local disk. Sends a *heartbeat* to the NameNode every 3 seconds.
Client	Reads/writes files via the NameNode (metadata) then communicates directly with DataNodes (data).

YARN – Resource Management



ResourceManager: allocates cluster resources globally.
NodeManager: manages resources on a single node.
ApplicationMaster: coordinates the execution of one job.

MapReduce – Overview



MapReduce – The 3 Phases in Detail

MAP Phase

- Reads data **line by line**
- Emits `key → value` pairs
- Runs in **parallel** on each node
- No communication between mappers

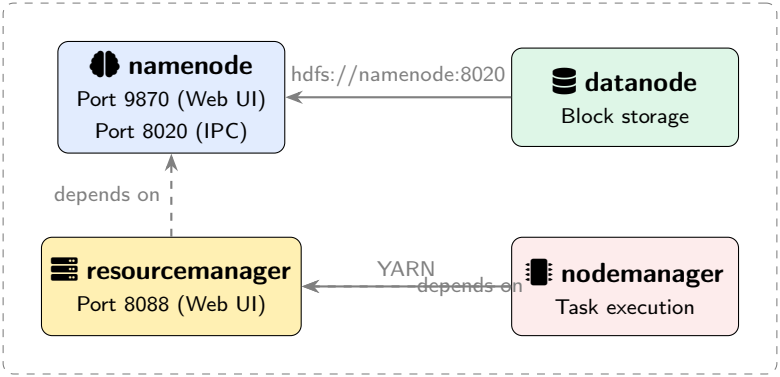
SHUFFLE Phase

- **Automatic** (handled by Hadoop)
- Sorts pairs by key
- Groups values by key
- Transfers data between nodes

REDUCE Phase

- Receives `key → [v1, v2, ...]`
- **Aggregates** values (sum, max, avg...)
- Writes final result to HDFS

Our Cluster – docker-compose



docker-compose.yml

```yaml
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    environment:
      - CLUSTER_NAME=test
      - CORE_CONF_fs_defaultFS=hdfs://namenode:8020
    ports:
      - "9870:9870"       # HDFS Web UI

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://namenode:8020

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    ports:
      - "8088:8088"       # YARN Web UI

  nodemanager:
    image: bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8
```

>_ Start: `docker compose up -d` | Stop: `docker compose down`

Practical Case – Objective

Scenario

A web server generates thousands of access log lines.
We want to analyze these logs to extract meaningful statistics.

Questions to answer:

1. How many requests per **HTTP status code**? (200, 404, 500...)
2. Which are the **most visited pages**?
3. What is the distribution by **HTTP method**? (GET, POST...)
4. How does **traffic vary by hour**?

Tools used

HDFS to store the logs + **MapReduce** (Hadoop Streaming + awk) to analyze them

Data Format

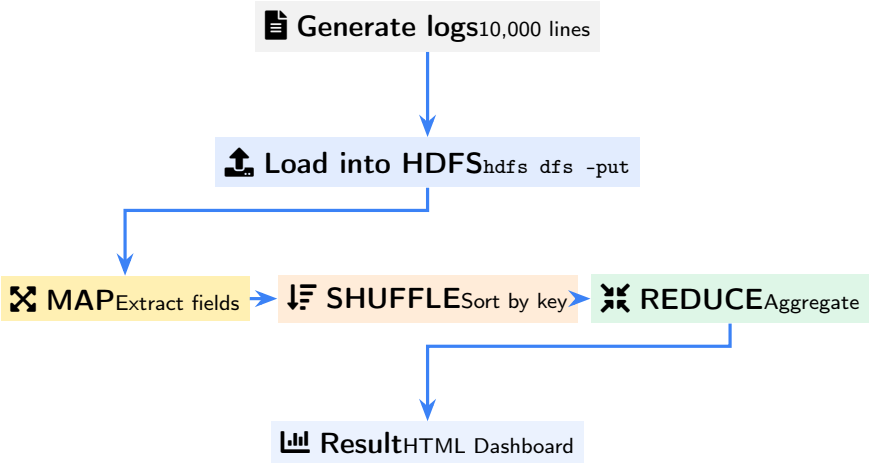
Each log line follows the **Apache Combined Log** format:

```
192.168.1.10 - - [25/Mar/2026:10:00:01] "GET /index.html HTTP/1.1" 200 1024
192.168.1.12 - - [25/Mar/2026:10:00:03] "GET /missing.html HTTP/1.1" 404 512
192.168.1.15 - - [25/Mar/2026:10:00:07] "GET /api/data HTTP/1.1" 500 64
```

Field breakdown (space-separated):

Field	Name	Example	Usage
$1	IP address	192.168.1.10	–
$4	Date/Time	[25/Mar/2026:10:00:01]	Hourly traffic
$5	Method	"GET	GET/POST distribution
$6	Page	/index.html	Most visited pages
$8	HTTP code	200	Response codes
$9	Size	1024	Transferred volume

Mechanism – Processing Pipeline



The Mapper (awk)

The mapper reads each line and emits key<TAB>value pairs:

```
BEGIN { OFS = "\t" }
{
    code = $8                # HTTP code (200, 404, 500...)
    if (code ~ /^[0-9]+$/) {
        print code, 1        # Emits: 200<TAB>1
    }
}
```

Input (1 log line):

```
192.168.1.10 - - [25/Mar/2026:10:00:01] "GET /index.html HTTP/1.1" 200 1024
```

Output:

```
200     1
```

The Reducer (awk)

The reducer receives pairs **sorted by key** and sums the counters:

```
BEGIN { FS = "\t"; OFS = "\t" }
{
    if ($1 == prev) {
        count += $2        # Same key: accumulate
    } else {
        if (prev != "") print prev, count
        prev = $1          # New key
        count = $2
    }
}
END { if (prev != "") print prev, count }
```

Input (sorted by Hadoop):

```
200     1
200     1
200     1
404     1
404     1
```

Running the Job

```
hadoop jar hadoop-streaming-3.2.1.jar \
    -D stream.num.map.output.key.fields=2 \
    -files /tmp/mapper.awk,/tmp/reducer.awk \
    -mapper  "awk -f /tmp/mapper.awk" \
    -reducer "awk -f /tmp/reducer.awk" \
    -input  /user/root/input/access.log \
    -output /user/root/output/results
```

What Hadoop does:

1. Splits the input file into **input splits**
2. Runs the **mapper** on each split (in parallel across DataNodes)
3. **Sorts and groups** key-value pairs (shuffle)
4. Runs the **reducer** to aggregate results
5. Writes output to HDFS (`part-00000`)

Results – HTTP Status Codes

Code	Requests
200 OK	5,846
301 Redirect	812
403 Forbidden	841
404 Not Found	809
500 Server Error	835
502 Bad Gateway	857
Total	**10,000**

Interpretation:

- ~58% successful (200)
- ~17% server errors (5xx)
- ~8% not found (404)
- ~8% access denied (403)

Action Item

500/502 error rates are high → investigate endpoints `/api/crash` and `/api/timeout`.

Results – Pages and Methods

Top 5 most visited pages:

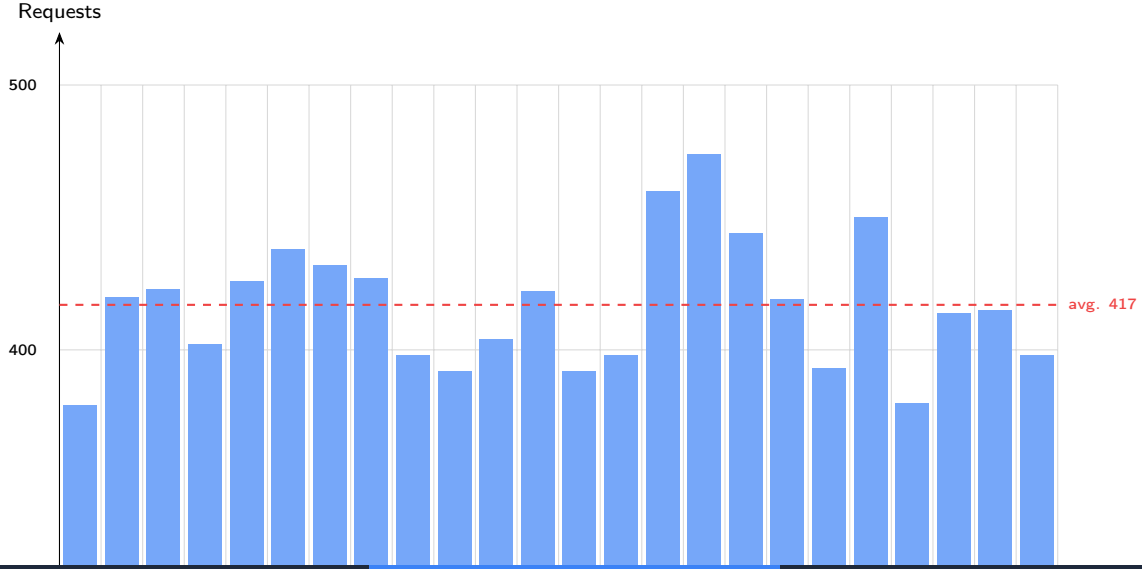
Page	Hits
/login	439
/deprecated	431
/search	423
/api/timeout	418
/blog/post-2	416

HTTP method distribution:

Method	Requests
GET	5,757 (57.6%)
POST	1,438 (14.4%)
PUT	1,412 (14.1%)
DELETE	1,393 (13.9%)

Traffic is mostly GET (read-heavy).

Results – Hourly Traffic



Monitoring Interfaces

HDFS – `localhost:9870`

- Cluster status
- Capacity and usage
- Active datanodes
- HDFS file browser

YARN – `localhost:8088`

- Application list
- Job status (SUCCEEDED, FAILED)
- Number of mappers / reducers
- Execution time and logs

Generated Dashboard – `localhost:8888`

The `run-cas-pratique.sh` script automatically generates an interactive HTML dashboard with Chart.js graphs: doughnut (codes), bar (hours, pages), pie (methods).

Key Takeaways

Hadoop enables:

- Distributed storage of massive data (**HDFS**)
- Parallel data processing (**MapReduce**)
- Cluster resource management (**YARN**)
- Automatic fault tolerance

In our practical case:

- 10,000 logs analyzed in minutes
- 4 dimensions extracted (codes, pages, methods, hours)
- Reproducible pipeline: single script
- Results viewable in the browser

Single command: `bash run-cas-pratique.sh` → Data → HDFS → MapReduce → Das

Thank you!

Questions?

- `github.com/Achaire-Zogo/hadoop`
- Dashboard: `http://localhost:8888/dashboard.html`
- `bash run-cas-pratique.sh`